Edinburgh Napier UNIVERSITY

Maps and Formats

Dr Peter Chapman

SET09123 – Interactive Data Visualisation

Lecture Summary

- Geography with D3
- Formatting Data
  - Numbers
  - Time

D3 Geographies

Visualising Maps

Edinburgh Napier UNIVERSITY

- With the d3-geo module, D3 comes with its own set of geography functionalities to help render geographical data onto our visualisations
  - github.com/d3/d3-geo

- Rendering paths
- Using known projections
- Making our own projections
- Spherical algebra and geometry

Visualising Maps

Edinburgh Napier UNIVERSITY

- With the d3-geo module, D3 comes with its own set of geography functionalities to help render geographical data onto our visualisations
  - github.com/d3/d3-geo

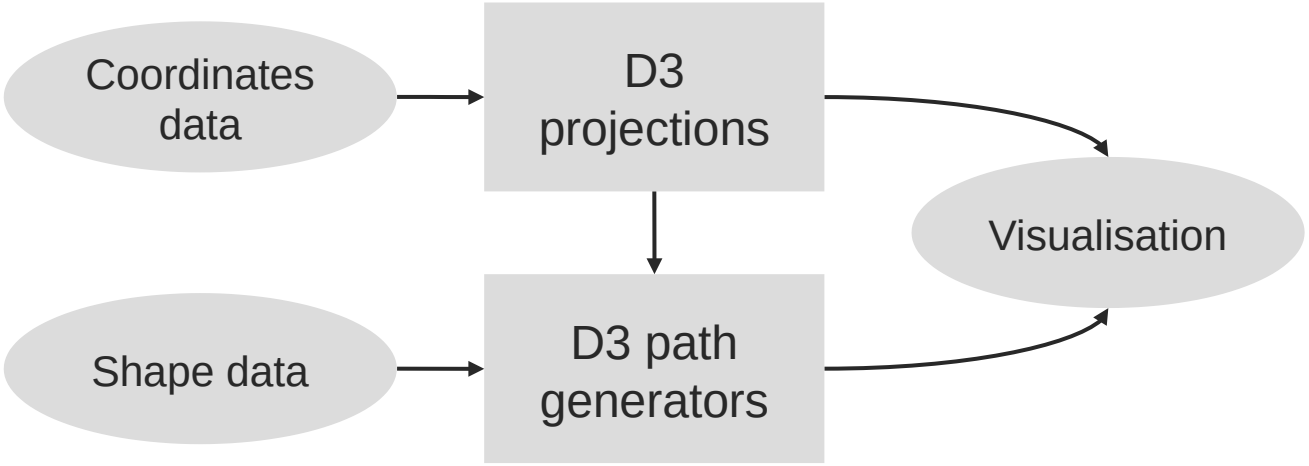
- **Rendering paths**
- **Using known projections**
- Making our own projections
- Spherical algebra and geometry

Using D3 Geographies

- Projection functions are first instantiated to map from geographical coordinates (latitude and longitude) to pixels
  - Similar to the d3-scale module

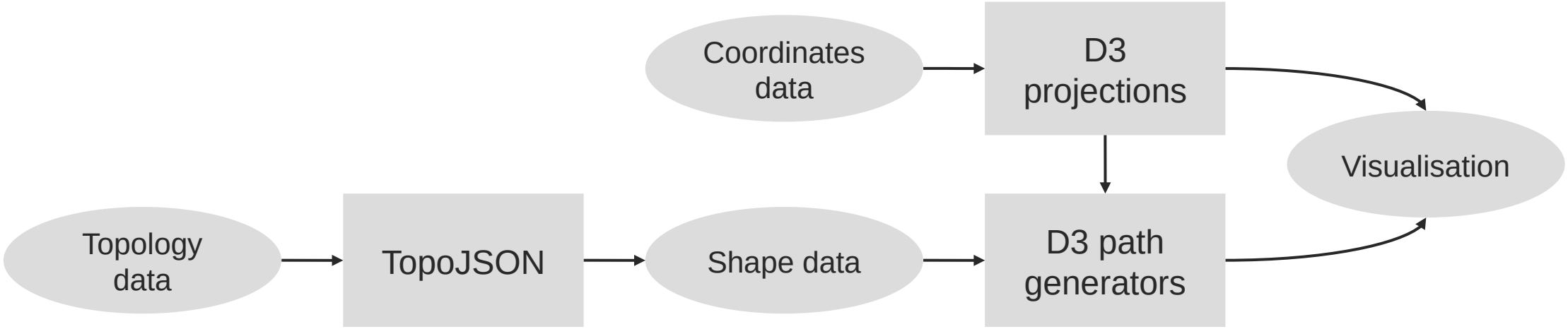
Using D3 Geographies

- Path Generators are there to help draw various geographical shapes
  - Similar to the d3-shape module
- They take the projection function as input to correctly transform the shapes



TopoJSON

- We also need to input topology data where the information about geography shapes is stored: lands, borders, etc.
- TopoJSON (an extension of GeoJSON) is a data format for such information



A Map of the UK

- First we must load the UK topology data and use TopoJSON to get the relevant components

```
import UK from '../uk.js';

let ukUnits = topojson.feature(UK, UK.objects.subunits),
    ukCities = topojson.feature(UK, UK.objects.places),
    ukBorders = topojson.mesh(UK, UK.objects.subunits, (a,b)=>a!==b),
    ukCoast = topojson.mesh(UK, UK.objects.subunits, (a,b)=>a==b);
```

A Map of the UK

- Then we need to declare a projection function
- D3 has already has a predefined list

```
let projection =
d3.geoConicConformal()
    .fitSize([600, 600], ukUnits);
```

- We can translate and scale the projection to better fit our view
- Or we use the `.fitSize` method to let D3 automatically adjust those factors

A Map of the UK

- The final thing we need is a path generator

```
let path = d3.geoPath()
    .pointRadius(4)
    .projection(projection);
```

- The `.pointRadius` method lets us define how big will points be when rendering landmarks

A Map of the UK

- Rendering the land

```
svg.selectAll('path.unit')
    .data(ukUnits.features)
    .join('path')
    .classed('unit', true)
    .attr('d', path)

  .style('fill',d=>d.id=='IRL'?'white':
'lightgrey');
```

A Map of the UK

- Rendering the borders

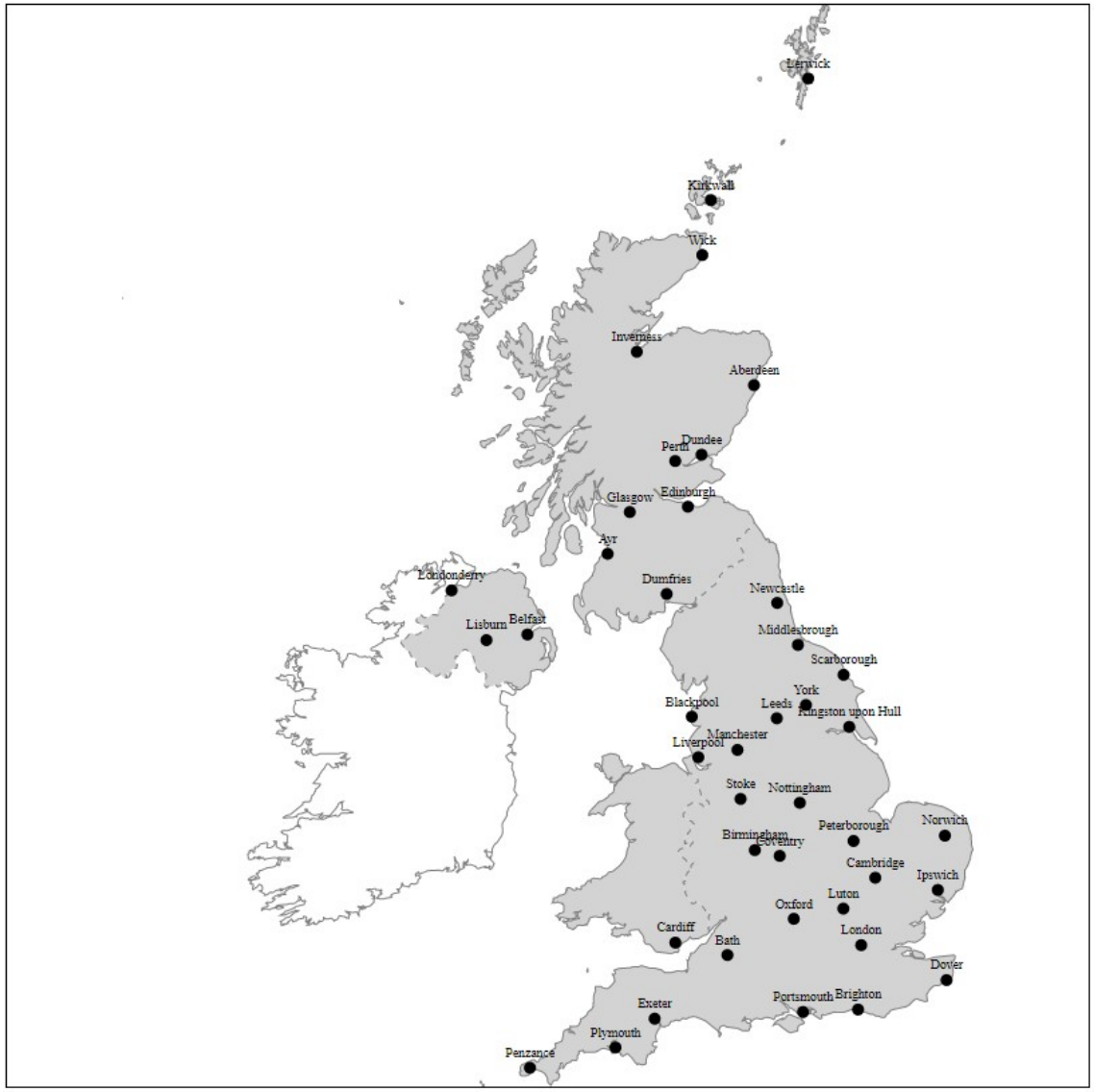
```
svg.selectAll('path.borders')
    .data([ukBorders])
    .join('path')
    .classed('borders', true)
    .attr('d', path)
    .style('fill', 'none')
    .style('stroke', 'grey')
    .style('stroke-dasharray', '5,5');
svg.selectAll('path.coast')
    .data([ukCoast])
    .join('path')
    .classed('coast', true)
    .attr('d', path)
    .style('fill', 'none')
    .style('stroke', 'grey');
```

A Map of the UK

- Rendering cities

```
svg.selectAll('path.city')
    .data([ukCities])
    .join('path')
    .classed('city', true)
    .attr('d', path);
svg.selectAll('text.city')
    .data(ukCities.features)
    .join('text')
    .classed('city', true)
    .attr('transform', d=>`translate($
{projection(d.geometry.coordinates)})`)
    .attr('dy', -8)
    .text(d=>d.properties.name)
    .style('font-size', '0.6em')
    .style('text-anchor', 'middle');
```

A Map of the UK



Formatting Data

Formatting

- In some instances, we might want to *format* the data we present to users, to make it more readable
- *1200345 → 1.2M*
- *2022-10-03T15:26:34.574Z → Oct. 2022*

Formatting Numbers

- The d3-format module (github.com/d3/d3-format) lets us define formatting functions for numbers

- Sign
- Symbol
- Grouping
- Precision
- Trailing zeros
- Notation type

Formatting Numbers

Edinburgh Napier UNIVERSITY

- The d3-format module (github.com/d3/d3-format) lets us define formatting functions for numbers

- Sign
- Symbol
- Grouping
- Precision
- Trailing zeros
- Notation type

  - - → minus sign for negative numbers only
  - + → plus sign for positive numbers (including 0) and minus sign for negative numbers
  - → space for positive numbers (including 0) and minus sign for negative numbers

Formatting Numbers

- The d3-format module (github.com/d3/d3-format) lets us define formatting functions for numbers

- Sign
- Symbol
- Grouping
- Precision
- Trailing zeros
- Notation type

  - $ → currency symbol (based on locale definition)

Formatting Numbers

Edinburgh Napier UNIVERSITY

- The d3-format module (github.com/d3/d3-format) lets us define formatting functions for numbers

- Sign
- Symbol
- Grouping
- Precision
- Trailing zeros
- Notation type

- `,` → to group thousands together, separated by a comma

Formatting Numbers

- The d3-format module (github.com/d3/d3-format) lets us define formatting functions for numbers

- Sign
- Symbol
- Grouping
- Precision
- Trailing zeros
- Notation type

- `.[number]` → set the number of significant digits or number of decimal points (for fixed point or percentage notation)

Formatting Numbers

Edinburgh Napier UNIVERSITY

- The d3-format module (github.com/d3/d3-format) lets us define formatting functions for numbers

- Sign
- Symbol
- Grouping
- Precision
- Trailing zeros
- Notation type

  - ~ → remove insignificant trailing zeros

Formatting Numbers

Edinburgh Napier UNIVERSITY

- The d3-format module (github.com/d3/d3-format) lets us define formatting functions for numbers

- Sign
- Symbol
- Grouping
- Precision
- Trailing zeros
- Notation type

- `e` → exponential notation
- `f` → fixed point notation
- `r` → decimal notation rounded to significant digit
- `s` → decimal notation with SI prefix (e.g., k for kilo, M for mega)
- `%` → percentage (multiply by 100 and add %)
- `d` → decimal notation rounded to integer

Usage

- `d3.format(options)` creates a new formatter
- `format(number)` returns the formatted number as a string
- `d3.formatLocale(options)` lets you create your own locale

Parsing and Formatting Dates

- Dates (and times) are typically stored as strings in datasets, e.g., “02/10/2021”
- But JavaScript (and D3) compute time with Date objects
- Hence, we must often parse strings to Date objects, and vice-versa

Parsing Dates

- The d3-time-format module provides a neat function for this
  - github.com/d3/d3-time-format

- `d3.timeParse` generates a date parser, given a date and time format

- The parser can then read strings and return Date objects

```
let parser = d3.timeParse('%d %B %Y');

let date = parser('04 September 2020');
```

Formatting Dates

- When displaying dates (or times), we then need to use a date formatter
- A date formatter does the reverse of a date parser, it converts Date objects to readable text
- `d3.timeFormat` generates a new formatter, given a date and time format to use

```
let formatter = d3.timeFormat('%b. %Y');

let date = formatter(new Date()); // 'Oct. 2022'
```

Date Formats

- Check the documentation to get a full list of the date and time format options:

  github.com/d3/d3-time-format

With Axes

- The `.ticks` and `.tickFormat` methods of axes allows you to set the format in which tick marks are printed
- Use number formatters or date formatters for this

Conclusion

Key Points

- To visualise geographical data, we use projections to transform geographical coordinates to pixel coordinates
- TopoJSON is a format for storing topological data: the shapes used in maps
- A path generator is used, with a projection, to render maps

- D3 provides tools to format data for users
  - Number formatters to print complex numbers in digestible formats
  - Time parsers to parse string to Date objects
  - Time formatters to print Date objects as readable strings